Reutilização, Propagação e Evolução de Agent Skills no GitHub

Jemison dos Santos
Universidade Estadual de Maringá – PR, Brasil
jemison321@gmail.com

***Resumo*—Contexto: Agent Skills são artefatos textuais reutilizáveis que descrevem capacidades, instruções e recursos para agentes de software. Sua cópia entre repositórios pode produzir divergência e alterações com possíveis efeitos comportamentais. Objetivo: Caracterizar a distribuição do reúso exato, a evolução posterior à cópia e mudanças que introduzem ou removem comandos, acesso externo e artefatos executáveis. Métodos: Analisamos um snapshot do GitSkills com hashes de conteúdo, métricas textuais, estruturais e contextuais. Reconstruímos históricos Git em uma amostra estratificada condicionada à recuperabilidade, comparamos diffs pai–filho e aplicamos classificadores determinísticos em cenários estrito e amplo. Resultados: Entre 1.586.188 conteúdos distintos, 16,02% ocorreram em pelo menos dois repositórios e o reúso respondeu por 42,06% das colocações adicionais. Seis de 80 ocorrências com âncora resolvida divergiram posteriormente. A busca inter-repositórios encontrou apenas um candidato de baixa confiança, sem denominador para estimar propagação. Em 75 comparações pós-âncora, as contagens comportamentais dependeram fortemente das regras; apenas 18 repositórios contribuíram com eventos, inviabilizando o modelo multivariado planejado. Conclusões: O reúso é concentrado, mas identidade, similaridade, evolução e transmissão exigem evidências distintas. Os resultados longitudinais são exploratórios e não sustentam inferência causal.**

***Index Terms*—engenharia de software empírica, agentes de software, reúso, mineração de repositórios, evolução de software**

I. Introdução

Arquivos de Agent Skills encapsulam instruções, exemplos, comandos e referências que podem ser incorporados a diferentes agentes e repositórios. A especificação define uma pasta centrada em `SKILL.md`, opcionalmente acompanhada por scripts e referências [2]. A facilidade de copiar esses artefatos cria um ecossistema de reúso, mas também introduz questões sobre divergência, atualização e possível propagação de alterações. Contar arquivos semelhantes não resolve essas questões: identidade byte a byte, similaridade textual, ancestralidade e transmissão temporal são relações empiricamente distintas.

Este trabalho analisa o percurso que vai da cópia exata à evolução observada. O estudo responde às seguintes questões:

- **RQ1:** Como a reutilização está distribuída e quais características textuais, estruturais e contextuais estão associadas ao seu alcance?
- **RQ2:** Quais padrões de estabilidade, divergência e propagação de alterações emergem após as skills serem copiadas entre repositórios?
- **RQ3:** Quais características estão associadas a mudanças que introduzem ou removem comandos, acesso externo ou artefatos executáveis?

As contribuições são: (i) uma caracterização populacional do reúso exato; (ii) um protocolo reprodutível para reconstruir evolução pós-âncora sem atribuir origem ao repositório representante; (iii) classificadores automatizados com cenários estrito e amplo; e (iv) uma demonstração empírica dos limites de inferência quando a história recuperável e o número de clusters são pequenos.

II. Trabalhos Relacionados

O GitSkills introduz um dataset de arquivos `SKILL.md` coletados de repositórios públicos, preservando ocorrências, hashes, conteúdo representativo, composição da pasta, metadados e parte do histórico de commits [1]. O próprio dataset propõe investigar concentração do reúso, genealogias, manutenção e mudanças que introduzem comandos ou acesso à rede. Este trabalho operacionaliza essas questões sobre uma população analítica mais restrita, definida por nome exato, conteúdo recuperado, hash validado e front matter válido.

Destefanis argumenta que skills devem ser tratadas como artefatos de software versionáveis, com interface, implementação, dependências e modos de falha [3]. Essa perspectiva fundamenta a separação entre propriedades textuais, estrutura do pacote e candidatos comportamentais. Ainda assim, detectar texto compatível com comandos ou rede não demonstra que uma ação foi executada.

Estudos de clones de código distinguem mudanças consistentes e inconsistentes e mostram que sua interpretação depende do histórico e da granularidade observada [4], [5]. Adotamos dessa literatura a preocupação com genealogia, ordem temporal e propagação de mudanças. A analogia é limitada: skills contêm linguagem natural e recursos heterogêneos; portanto, similaridade textual é usada apenas para recuperar candidatos e nunca como prova de ancestralidade.

III. Método

A. Conceitos e níveis de evidência

Para evitar que proximidade textual seja interpretada indevidamente como evidência de transmissão, distinguimos quatro conceitos ao longo da análise. A identidade de conteúdo é estabelecida pela igualdade do hash do blob, enquanto a similaridade serve apenas à recuperação de candidatos e, isoladamente, não comprova ancestralidade. A evolução, por

sua vez, corresponde a mudanças pai–filho observadas no grafo Git de uma ocorrência. A caracterização de propagação exige evidência adicional: uma alteração comparável, ordem temporal, repositórios distintos e oportunidade de observação. Consequentemente, o repositório representante empregado na deduplicação não é presumido como a origem do conteúdo.

Com base nessa distinção, organizamos as relações genealógicas em quatro níveis de evidência, de A a D. O nível A, o mais forte, requer proveniência explícita ou ancestralidade Git compartilhada. Na ausência dessa evidência direta, uma relação é classificada no nível B quando apresenta modificação exata, pertence à mesma família-âncora, reproduz uma transição idêntica entre blobs não vazios e respeita ordem temporal estrita. O nível C abrange candidatos temporalmente compatíveis, mas sustentados por evidência menos específica; por fim, os casos em que não é possível estabelecer direção ou reunir evidência suficiente permanecem no nível D.

B. População e análise da RQ1

A população foi extraída de um snapshot local do GitSkills em SQLite [1]. Foram mantidos registros representantes da deduplicação, com nome `SKILL.md`, conteúdo recuperado, hash validado e front matter válido. A unidade foi o conteúdo distinto identificado por hash. O reúso foi definido como o número de repositórios distintos contendo esse hash.

Foram modelados dois desfechos: ocorrência de reúso, por regressão logística, e quantidade de reutilizações adicionais entre conteúdos reutilizados. A binomial negativa truncada inicialmente planejada produziu solução degenerada em implementações independentes; por isso, a intensidade foi estimada por Poisson de pseudomáxima verossimilhança. Ambos os componentes utilizaram erros-padrão agrupados pelo repositório representante. Variáveis numéricas assimétricas foram transformadas por `log1p`, winsorizadas nos percentis 0,5 e 99,5 e padronizadas. O controle de falsos achados empregou Benjamini–Hochberg separadamente em cada componente.

C. Amostragem e recuperação histórica

O frame longitudinal partiu dos hashes exatos presentes em pelo menos dois repositórios, preservando todas as ocorrências repositório–path. A sondagem de acesso anônimo foi instável; portanto, a análise subsequente foi explicitamente condicionada aos 603 repositórios classificados como acessíveis em duas sondagens. Foram sorteados cinco repositórios em cada um de 11 estratos primários mutuamente exclusivos, com seed 20261026, totalizando 55 repositórios e 122 ocorrências.

Dois repositórios tiveram coleta incompleta. Para evitar mistura de linhas parciais, todas as suas 40 ocorrências foram excluídas sem reposição. Restaram 82 ocorrências em 53 repositórios completos. Probabilidades e pesos corrigem apenas a seleção dentro do frame condicionado; não corrigem não resposta, instabilidade da sondagem ou cobertura do GitHub.

D. Reconstrução da evolução

Para cada ocorrência, a âncora temporal foi o primeiro timestamp de commit observado em que o blob coincide com o hash exato do snapshot. Somente comparações com timestamp estritamente posterior foram classificadas como pós-âncora. Cada pai de merge foi preservado como comparação distinta. Diffs unificados registraram paths, blobs, churn e operação estrutural: adição, modificação, movimento ou remoção.

Um classificador determinístico separou front matter, títulos, parágrafos, listas, código, tabelas, citações e linhas vazias. A inferência longitudinal foi limitada a casos com âncora resolvida. Proporções não ponderadas receberam intervalos de Wilson; estimativas condicionais ponderadas receberam intervalos por bootstrap estratificado com reamostragem de repositórios e 2.000 réplicas.

E. Busca de propagação entre repositórios

Patches foram normalizados por Unicode NFC, remoção de espaços periféricos e colapso de espaços internos; a assinatura SHA-256 preservou operação, direção e ordem das linhas. Movimentos puros e patches vazios foram excluídos. A busca exigiu mesma família de hash-âncora, assinatura idêntica, divergência pós-âncora e repositórios diferentes. Repetições em paths ou commits do mesmo repositório foram colapsadas na primeira adoção observada. Igualdade de assinatura foi tratada como mecanismo de recuperação, não como prova de transmissão.

F. Classificação e análise da RQ3

As regras estritas reconheceram comandos conhecidos em regiões de código, tokens operacionais de rede, invocações explícitas de scripts e siblings com modo executável ou shebang. As regras amplas acrescentaram linhas com forma de comando, URLs explícitas, menções a paths de scripts e extensões executáveis. Um evento pôde receber múltiplos rótulos.

A base associativa contém somente covariáveis anteriores ao evento ou marcadas explicitamente como concorrentes. As características anteriores incluem idade desde a âncora, eventos e divergências anteriores, intensidade de reúso, classe do path e conteúdo do blob pai. Churn, merge e operação são concorrentes e não são tratados como causas antecedentes. Características do pai foram avaliadas somente nos 48 eventos com blob pai.

Antes dos modelos foi aplicado um gate: ao menos 50 eventos, dez positivos, cinco repositórios positivos, cinco negativos e 30 clusters. Quando o gate falhasse, o protocolo determinava análise descritiva. Razões de risco ponderadas e intervalos bootstrap foram produzidos apenas como contrastes exploratórios, sem valores de p, FDR ou interpretação causal.

G. Validação e reprodutibilidade

Por restrição operacional, não houve anotação humana independente. As categorias de RQ2 e RQ3 são definições automatizadas, não estimativas de acurácia semântica. A validação empregou fixtures, invariantes, taxonomia explícita de falhas, cenários estrito/amplo, manifests com checksums e reexecuções em diretórios novos. Trinta e dois testes automatizados foram aprovados; os artefatos finais das etapas longitudinais, de propagação e de associação foram reproduzidos byte a byte.

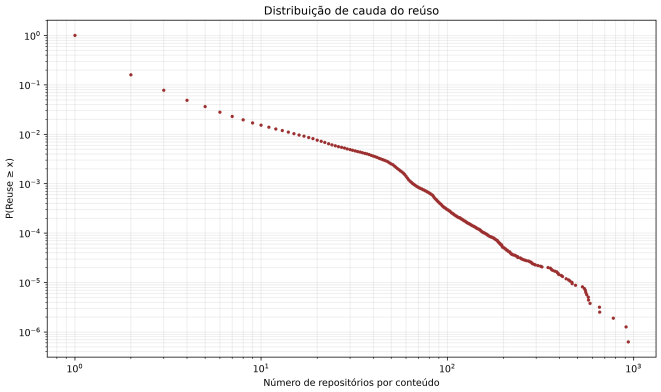


Figura 1. Distribuição complementar do reúso exato. O eixo log–log evidencia a cauda longa.

Tabela I
ASSOCIAÇÕES PRINCIPAIS DA RQ1 APÓS FDR

Característica	Probabilidade	Intensidade
Descrição	0,924	0,867
Legibilidade Flesch	0,778	0,903
URLs	–	1,075
Blocos de código	–	0,920
Cabeçalhos	1,241	1,106
Arquivos irmãos	1,121	–
Estrelas	–	0,838
Inatividade	1,223	–

IV. RESULTADOS

A. RQ1: distribuição e fatores associados ao reúso

Foram observados 1.586.188 conteúdos distintos. Destes, 1.332.110 (83,98%) apareceram em um único repositório e 254.078 (16,02%) em pelo menos dois. A mediana foi um repositório, a média 1,73, o máximo 937 e o Gini 0,398. Embora minoritário entre conteúdos distintos, o reúso respondeu por 42,06% das colocações adicionais.

A Tabela I resume os efeitos principais que permaneceram após correção FDR. As razões correspondem a um desvio-padrão na variável transformada, exceto indicadores binários.

Os erros agrupados foram, na mediana, 10,95 vezes maiores que os erros HC1, mostrando a importância da dependência intrarrepositório. Após remover 16 megarepositórios, as 28 associações das 14 características principais mantiveram direção. A winsorização da cauda positiva também preservou direção e classificação de significância dos 37 coeficientes de intensidade. Esses resultados são associações observacionais, não mecanismos causais de adoção.

B. RQ2: estabilidade, divergência e propagação

Das 82 ocorrências elegíveis, 80 tiveram âncora resolvida. Seis divergiram posteriormente: 7,5% sem ponderação (IC95% de Wilson: 3,5–15,4%) e 1,0% na estimativa ponderada condicional (bootstrap: 0,3–1,9%). Foram observadas 75 comparações pós-âncora: 27 adições, 24 modificações, 19 movimentos e cinco remoções. A concentração em poucos repositórios produziu intervalos amplos para a composição ponderada dos eventos.

Tabela II
CANDIDATOS COMPORTAMENTAIS EM 75 COMPARAÇÕES

Alvo e direção	Estrito	Amplo
Comando adicionado	11	40
Comando removido	5	16
Acesso externo adicionado	3	9
Acesso externo removido	1	1
Referência a script adicionada	1	21
Referência a script removida	4	11
Sibling executável adicionado	1	1
Sibling movido/copiado	0	1

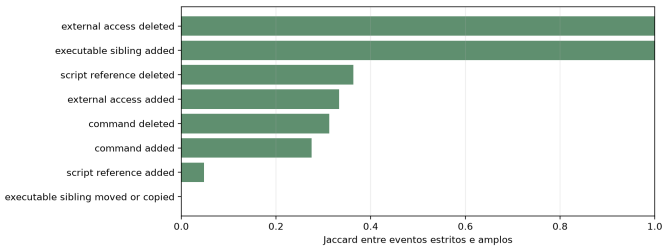


Figura 2. Sobreposição entre as definições estrita e ampla. Categorias sem eventos nos dois cenários foram omitidas.

A assinatura foi calculável para 151 dos 173 eventos de mudança do corpus histórico completo. Dezenove grupos apareciam em mais de um repositório antes dos filtros substantivos. Após exigir divergência pós-âncora, mesma família, ordem temporal e colapso por repositório, restou uma relação candidata: remoção do mesmo conteúdo observada cerca de 77,12 dias depois em outro repositório. Por ser uma remoção de baixa especificidade, recebeu confiança C. Nenhuma relação alcançou A ou B, e nenhum repositório foi denominado origem.

Não foi possível estimar uma taxa de propagação: a janela temporal mostra oportunidade de observação, mas não demonstra que o contexto ao qual o patch se aplicaria permanecia presente nas linhagens não adotantes. Assim, a evidência sustenta divergência intrarrepositório, mas não transmissão inter-repositórios.

C. RQ3: candidatos comportamentais e associações

A Tabela II apresenta os desfechos com maior frequência. As diferenças entre cenários são substantivas. O Jaccard entre eventos estritos e amplos foi 0,275 para comandos adicionados, 0,333 para acesso externo adicionado e somente 0,048 para referências a scripts adicionadas.

No cenário amplo, 16 dos 40 eventos que adicionaram comandos também adicionaram referências a scripts; os nove candidatos de adição de acesso externo também receberam o rótulo de comando adicionado. A coocorrência pode decorrer da mesma linha acionando regras distintas e não implica mecanismos independentes.

Somente 18 repositórios contribuíram com eventos, e um deles concentrou 29 das 75 comparações. Nenhum desfecho

passou o gate para regressão multivariada. Entre os 280 contrastes não ajustados, idade desde a âncora, intensidade de reúso, conteúdo do pai e classe de localização apresentaram diferenças descritivas. Entretanto, os contrastes foram sensíveis aos pesos e à concentração; características concorrentes como churn e merge também aumentam mecanicamente a oportunidade de detecção. A resposta defensável à RQ3 é, portanto, descritiva: não há suporte para identificar características independentemente associadas aos desfechos.

V. Discussão

Em conjunto, as RQs mostram que alcance e linhagem não são intercambiáveis. A RQ1 evidencia que conteúdos idênticos podem alcançar muitos repositórios, mas o hash compartilhado não informa direção, mecanismo ou origem. A RQ2 acrescenta ordem temporal, porém a baixa divergência e a cobertura condicionada restringem a busca de transmissão. O candidato C não confirma propagação nem demonstra que ela inexiste.

A sensibilidade da RQ3 às regras é também um resultado metodológico: referências, URLs e trechos com forma de comando não equivalem a execução. Estudos futuros devem separar detecção sintática, interpretação semântica e comportamento observado. Na prática, registrar fonte declarada, versão e atualizações de skills copiadas facilitaria auditoria posterior [3]. Essa recomendação decorre dos limites de rastreabilidade, não de avaliação experimental de sua eficácia.

VI. Ameaças à Validade

A. Validade de construto

Hash exato captura identidade de bytes, não equivalência semântica. As regras RQ3 detectam texto e estrutura compatíveis com comportamento; não demonstram execução, acesso efetivo à rede ou impacto de segurança. A ausência de referência humana independente impede estimar precisão e revocação. As diferenças entre cenários estrito e amplo mostram que a taxonomia é uma fonte central de incerteza.

B. Validade interna

O desenho é observacional. Associações não são causais, timestamps não provam transmissão e a primeira ocorrência observada não é necessariamente a origem. Fontes comuns não observadas, cópias fora do Git e reescrita de histórico podem explicar relações compatíveis. Comparações de merges, eventos repetidos e múltiplos paths introduzem dependência; o pipeline preserva pais e colapsa episódios por repositório quando necessário, mas não elimina toda dependência.

C. Validade externa

A análise longitudinal está condicionada a 603 repositórios acessíveis em duas sondagens instáveis e, depois, a 53 repositórios completos. Quarenta ocorrências foram excluídas por falha de coleta. Os pesos não corrigem essas perdas nem autorizam generalização aos 91.529 repositórios do frame de âncoras ou ao GitHub. Repositórios privados, removidos ou inacessíveis não são observados.

D. Validade de conclusão

A RQ1 possui grande população, mas seus erros-padrão mudam substancialmente ao considerar clusters. Na RQ2, somente seis ocorrências divergiram e a busca de propagação encontrou um candidato C; não existe denominador contextual para uma taxa. Na RQ3, 18 clusters são insuficientes para o modelo planejado. Os contrastes descritivos não foram ajustados por confundidores ou multiplicidade e não devem ser lidos como significância estatística.

E. Reprodutibilidade

Scripts, seeds, versões de classificadores, manifests e checksums estão versionados; execuções independentes reproduziram as saídas finais. Entretanto, o banco original tem aproximadamente 44 GB e não é distribuído, e a recuperabilidade de repositórios públicos pode mudar. O checksum local identifica a cópia auditada, não um snapshot publicado por terceiro.

VII. Disponibilidade dos Artefatos

Os scripts, protocolos, matrizes de evidência e resultados derivados serão arquivados no Zenodo. O endereço persistente deverá ser inserido na versão final da disciplina: **[ADICIONAR DOI/URL DO ZENODO]**. O banco GitSkills original é obtido separadamente conforme as instruções de seus autores [1]; o depósito deste estudo não deve redistribuí-lo sem confirmação de licença e capacidade.

VIII. Conclusão

O reúso exato de Agent Skills é minoritário entre conteúdos distintos, mas representa parcela substancial das colocações observadas e possui distribuição de cauda longa. A evolução posterior à cópia foi rara na amostra condicionada e concentrou-se em poucos repositórios. A busca conservadora não encontrou evidência forte de propagação inter-repositórios, e sua taxa permaneceu não estimável. Mudanças compatíveis com comportamento dependeram fortemente da definição operacional, enquanto o número efetivo de clusters impediu modelos associativos ajustados. Esses achados reforçam que presença compartilhada, similaridade, ancestralidade e propagação não devem ser tratadas como sinônimos em estudos de ecossistemas de agentes.

Referências

[1] G. Destefanis, D. Graziotin, M. Vaccargiu e M. Ortu, "GitSkills: A Dataset of Agent Skills on GitHub," em *Proc. 24th Int. Conf. on Mining Software Repositories (MSR)*, 2027, a publicar. arXiv:2608.10906.

[2] Agent Skills, "Agent Skills Specification," especificação aberta, Apache-2.0. [Online]. Disponível: https://agentskills.io. Acesso em: 26 out. 2026.

[3] G. Destefanis, "Authoring Agent Skills: A Software-Engineering Approach," arXiv:2607.25032, 2026.

[4] J. Krinke, "A Study of Consistent and Inconsistent Changes to Code Clones," em *Proc. 14th Working Conf. on Reverse Engineering (WCRE)*, 2007, pp. 170–178.

[5] N. Bettenburg, W. Shang, W. M. Ibrahim, B. Adams, Y. Zou e A. E. Hassan, "An Empirical Study on Inconsistent Changes to Code Clones at the Release Level," *Science of Computer Programming*, vol. 77, pp. 760–776, 2012, doi: 10.1016/j.scico.2010.11.010.